

```
"""
```

```
data_feed.py
```

```
-----
```

```
Pluggable price-data fetcher. Default implementation uses yfinance  
(free, no API key). Swap get_ohlc() for your broker/data API of choice  
(OANDA, Twelve Data, Alpha Vantage, MetaTrader, etc.) -- just keep the  
same return shape: a DataFrame indexed by datetime with columns  
['Open', 'High', 'Low', 'Close'].
```

```
Install the default backend with:
```

```
    pip install yfinance
```

```
"""
```

```
import pandas as pd
```

```
def get_ohlc(pair: str, period: str = "6mo", interval: str = "1d") -> pd.DataFrame  
    """
```

```
    pair: e.g. 'EURUSD' (will be converted to yfinance's 'EURUSD=X' format)
```

```
    period: yfinance lookback window, e.g. '3mo', '6mo', '1y'
```

```
    interval: '1d', '4h' (via '1h' resample), '1h', etc.
```

```
    """
```

```
    try:
```

```
        import yfinance as yf
```

```
    except ImportError as e:
```

```
        raise ImportError(
```

```
            "yfinance is not installed. Run: pip install yfinance\n"
```

```
            "Or replace data_feed.get_ohlc() with your own broker/data API call,
```

```
            "keeping the same return shape (DataFrame indexed by datetime with "
```

```
            "Open/High/Low/Close columns)."
```

```
        ) from e
```

```
    ticker = f"{pair}=X"
```

```
    df = yf.download(ticker, period=period, interval=interval, progress=False, au
```

```
    if df.empty:
```

```
        raise ValueError(f"No data returned for {pair} ({ticker}). Check the symb
```

```
    if isinstance(df.columns, pd.MultiIndex):
```

```
        df.columns = df.columns.get_level_values(0)
```

```
    df = df[["Open", "High", "Low", "Close"]].dropna()
```

```
    return df
```

```

def generate_synthetic_ohlc(n: int = 180, seed: int = None, drift: float = 0.0002
                             vol: float = 0.006) -> pd.DataFrame:
    """
    Generates synthetic OHLC data for testing the pipeline offline (no
    network required). Not for real analysis -- only used to validate
    that the technical engine and cross-reference logic run correctly
    end-to-end before you point it at live data.
    """
    import numpy as np
    rng = np.random.default_rng(seed)
    returns = rng.normal(drift, vol, n)
    close = 1.10 * np.exp(np.cumsum(returns))

    high = close * (1 + np.abs(rng.normal(0, vol / 2, n)))
    low = close * (1 - np.abs(rng.normal(0, vol / 2, n)))
    open_ = np.roll(close, 1)
    open_[0] = close[0]

    idx = pd.date_range(end=pd.Timestamp.today(), periods=n, freq="D")
    return pd.DataFrame({"Open": open_, "High": high, "Low": low, "Close": close})

```